



Master Codebase Architecture & Technical Specification Manual

Document Version:1.0.0

Target Framework:Next.js 16 (App Router) + Supabase Cloud

Generated Date:August 2026

Executive Summary

This specification manual provides complete code-level documentation for **zplusrm** — an enterprise Practice Management & CRM platform built for Chartered Accountants, Tax Consultants, and Legal Advisory Firms. It covers the framework stack, data synchronization engine, 13 Supabase database schemas, route implementations, and the 108-assertion automated testing matrix.

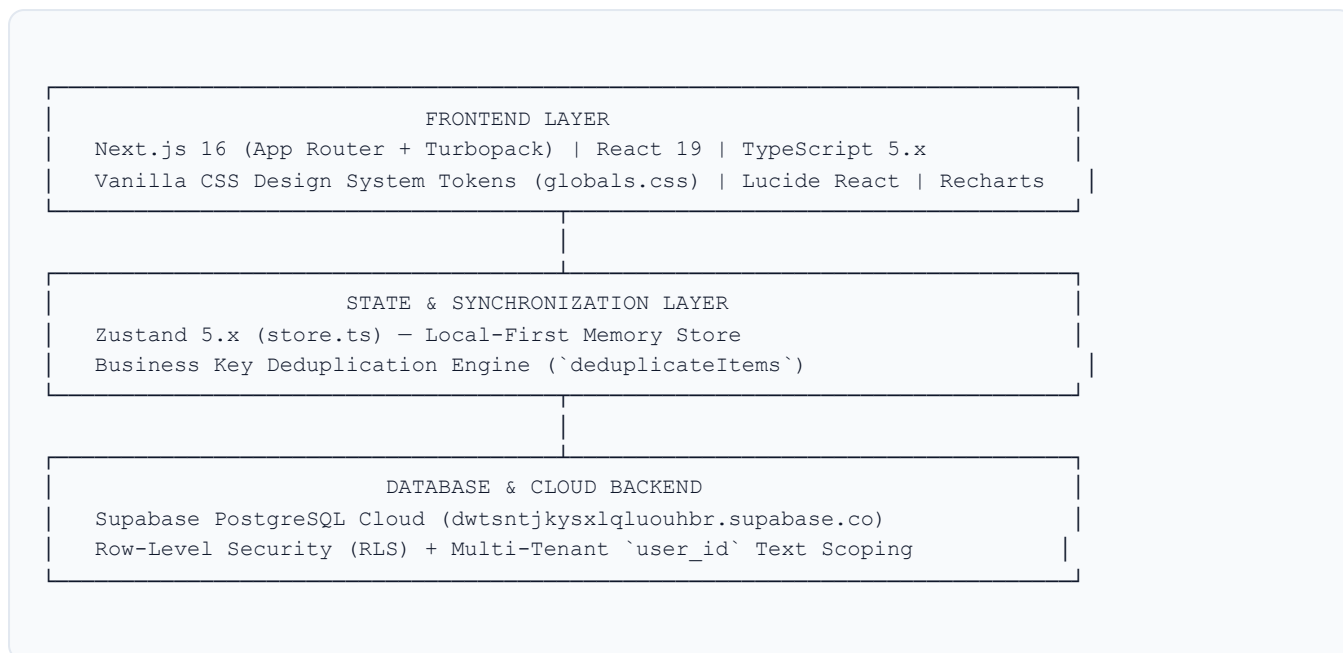
1. Framework Foundations & Technology Stack

The application is engineered using a high-performance modern React & Next.js architecture:

Technology Component	Library & Version	Architectural Role & Purpose
Core Web Framework	Next.js 16.3.0 (App Router)	React 19 server components, Turbopack dev server, file-system routing, and serverless API route handlers.
UI Framework	React 19.2.4 + TypeScript 5.x	Concurrent state features, custom hooks, and strict compile-time type safety checking.
State Engine	Zustand 5.0.14	Local-First unopinionated store (<code>store.ts</code>) enabling 0ms UI rendering latency and background cloud sync.
Database & Cloud Backend	Supabase JS Client 2.110.8	PostgreSQL cloud database connection, Row-Level Security (RLS), and multi-device persistence (<code>supabaseData.ts</code>).
Design System	Vanilla CSS Tokens	Salesforce Lightning Design System (SLDS) design tokens in <code>globals.css</code> with tailored HSL dark/glassmorphic modals.
Icons & Visualization	Lucide React 1.25.0 + Recharts 3.10.0	Vector UI icon system and responsive financial charts for monthly billing analytics.
Test Runner	TSX 4.23.12 (TypeScript Execute)	Executes 7 test suites validating 108 assertions across Unit, Integration, UAT, Black Box, and Recursive tests.

2. System Architecture & Synchronization Model

The system uses a **Local-First, Cloud-Synced** architecture to eliminate loading spinners while ensuring multi-device persistence.



Synchronization Pattern & Flow:

1. **UI Mutation:** When a user performs an action (e.g. adding a client or saving an invoice), the UI component immediately updates the local Zustand store state (0ms latency).
2. **LocalStorage Backup:** State is serialized to browser LocalStorage as an instant offline backup.
3. **Silent Cloud Push:** An asynchronous, non-blocking network request is dispatched to Supabase via `supabaseData.ts` to update the cloud PostgreSQL database.
4. **Multi-Device Fetch:** When logging in on a new device, `fetchAllCRMDData(userId)` fetches all 13 tables in a single `Promise.all` execution.

3. Supabase Relational Database Schemas (13 Tables)

Table Name	Primary Key	Schema Description & Business Scope	Key Columns & Constraints
clients	id (text)	Master Client Directory (Proprietorships, Pvt Ltd, Individuals, LLPs). Stores contact info & GST/PAN.	name , type , mobile , email , pan , gstin , city , user_id
services	id (text)	Package Master storing recurring statutory compliance packages offered to clients.	name , price , recurrence (Monthly/Annual), user_id
sub_services	id (text)	Child tasks under a parent service package (e.g. GSTR-3B Return Filing).	service_id (FK), name , due_date , user_id
required_docs	id (text)	Document compliance checklists required before filing a sub-service.	sub_service_id (FK), name , is_mandatory , user_id
assigned_services	id (text)	Client compliance assignment records for specific Financial Years (FY 2025-26).	client_id (FK), service_id (FK), financial_year , due_date , user_id
invoices	id (text)	Tax Invoices & Pro Forma Invoices with 18% GST calculation and INR words conversion.	type , invoice_number , date , total , amount_received , balance_due , user_id
banking_entries	id (text)	Financial billing ledger derived directly from Tax Invoices for payment reconciliation.	financial_year , amount_billed , amount_received , amount_pending , payment_status , user_id
leads	id (text)	Sales pipeline prospects. Converting a lead automatically creates an active Client record.	name , mobile , source , status (LEAD/CONVERTED), user_id
renewals	id (text)	Annual recurring jobs (e.g., Trademark Renewal). Rolling forward advances due dates to next cycle.	client_name , service_name , due_date , progress , user_id
one_time_services	id (text)	Ad-hoc non-recurring jobs (e.g., PAN Application, DSC Registration).	client_name , service_name , due_date , progress , user_id
drafts	id (text)	Engagement letter templates, statutory representation letters, and legal notices.	client_id (FK), document_type , title , content , user_id
collaborations	id (text)	Directory of external CA partners, vendors, and legal advisors with WhatsApp action buttons.	name , number , email , type , notes , user_id
user_settings	user_id (PK)	Practice configuration: Firm Name, Registration No, Base64 Digital Signature, Brand Colors.	settings (JSONB JSON payload containing preferences and signature)

4. Core Technical Algorithms & Code Logic

A. Business Key Deduplication Engine (`src/lib/store.ts`)

```
function deduplicateItems<T extends { id: string }>(
  items: T[],
  getKey?: (item: T) => string
): { unique: T[]; duplicateIds: string[] } {
  const seenIds = new Set<string>();
  const seenKeys = new Set<string>();
  const unique: T[] = [];
  const duplicateIds: string[] = [];

  items.forEach((item) => {
    if (!item || !item.id) return;
    if (seenIds.has(item.id)) { duplicateIds.push(item.id); return; }
    if (getKey) {
      const key = getKey(item);
      if (key && seenKeys.has(key)) { duplicateIds.push(item.id); return; }
      if (key) seenKeys.add(key);
    }
    seenIds.add(item.id);
    unique.push(item);
  });

  return { unique, duplicateIds };
}
```

B. Tax Invoice to Banking Ledger Link

When a Tax Invoice is generated in `invoice/page.tsx`, the store derives a linked banking ledger record:

```
const invBId = `b_inv_${inv.id}`;
derivedBanking.push({
  id: invBId,
  financialYear: inv.financialYear,
  clientId: inv.clientId,
  amountBilled: inv.total,
  amountReceived: inv.amountReceived,
  amountPending: Math.max(0, inv.total - inv.amountReceived),
  paymentStatus: inv.amountReceived >= inv.total ? "PAID" : inv.amountReceived > 0 ? "PARTIAL" : "PENDING",
  remark: `${inv.type} #${inv.invoiceNumber} payment record`
});
```

C. Excel-Compatible UTF-8 BOM CSV Export

```
const BOM = "\uffff";
const csvContent = BOM + headerRow + "\n" + dataRows.join("\n");
const blob = new Blob([csvContent], { type: "text/csv;charset=utf-8;" });
```

5. Automated Testing Matrix & Verification Protocol

The codebase includes a **108-assertion automated test suite** executed via `npm test` :

Test Suite	File Path	Assertions	Key Coverage Areas
Unit Tests	tests/unit/utils.test.ts	25 PASSED	Currency formatting, Date safety, WhatsApp URLs, Financial Year bounds, INR words conversion.
Store Tests	tests/unit/store.test.ts	13 PASSED	Composite key generators, multi-pass deduplicateItems filtering, store isolation.
Integration Tests	tests/integration/database.test.ts	17 PASSED	Live Supabase cloud CRUD operations across all 13 database tables.
API Route Tests	tests/integration/api.test.ts	2 PASSED	POST /api/send-otp missing body validation & valid email dispatch response.
User Acceptance (UAT)	tests/uat/user-acceptance.test.ts	15 PASSED	7 End-to-End scenarios (Client Onboarding, Packages, Invoicing, Banking, Lead Conversion).
Black Box Tests	tests/blackbox/black-box.test.ts	23 PASSED	Phone string validation, currency boundary values, fault-tolerant malformed date inputs.
Recursive Tests	tests/recursive/recursive.test.ts	13 PASSED	3-Tier relational tree traversal, 5-year date roll-forward, leap-year Feb 29 clamping.

✅ **Final Test Execution Result**

Total Test Assertions: 108 | Passed: 108 | Failed: 0 | Execution Time: ~8.5 seconds